

The Designer-Maker's Agentic Design Playbook

The age of the 'handoff' is dead. AI and MCP have dissolved the barriers between describing, designing, and building a product. Agents can now directly manipulate our design tools, write our code, and launch our products. The technological walls have fallen, and the artificial walls separating Experience, Product, and Engineering must fall too.

This is a playbook for the designer who intends to survive—and lead—what comes next.

For this to work you need to believe in:

1. Files > Tools

- *Why?* Proprietary design tools trap your work. You must be able to change tools without losing your work.
- *Therefore:* Code is the ultimate source of truth, not artboards. Through MCP, design tools become temporary lenses for our code, letting you 'see into the Matrix', rather than prisons for our creations.

2. Observation > Speed

- *Why?* AI unlocks scale, but generating UI *en mass* wastes time, compute and users' goodwill.
- *Therefore:* Deep human expertise in HCI, behavioural design, cognitive science and empathy prevent slop before it happens.

3. Making Things > Making Pictures

- *Why?* You can't run analytics or test accessibility on a static image. Making pictures of software pure waste. It's the antithesis of Lean UX.
- *Therefore:* We prototype in functional code. The canvas is another workspace for Agents, not a museum for mock-ups.

Workflows

Creating

1. **Generate and curate:** Sketch by hand. In your design tool,^[1] use agents to generate variations at scale. Scare yourself. Curate the results and teach the agent by turning your feedback into design and interaction specs.
2. **Visual refinement:** Finesse the design manually when it would be faster than reprompting.
3. **Canvas to code:** Use MCP to import your design using your chosen UI framework.^[2]
4. **Version control:** Keep variations in Git branches or IDE subfolders for testing.
5. **Behaviour and flow:** Write the user flow in BDD format, and prompt the Agent to turn these into QA test scripts.

6. **Prototype development:** The Agent writes the logic and (via MCP) builds any required tooling (eg BaaS or mock data). It then runs the tests to verify its own work before you review.
7. **Human polish:** Tweak the code in your IDE to polish the UI. Pair with a feasibility expert to ensure the prototype can be integrated into production with minimal rework.
8. **Instrumentation:** Add monitoring and product analytics^[3] code.
9. **Deploy:** Push the prototype to a testing environment.

Iterating

1. **Code-first iteration:** Pair with a feasibility expert to refine, and use Git to branch, tweak code, test, and merge.
2. **Code to canvas:** As needed, use Agents to push coded flows back into the design tool.
^[4] Edit visually, and repeat the pipeline.

Remixing

1. **Source the truth:** Use MCP to pull live components from the codebase into your design tool.
2. **Create flows:** Build new journeys using only existing components.
3. **Define behavior:** Write BDD specs for the new flows and generate test scripts.
4. **Build and verify:** Bring the designs back to code. The agent writes the connecting logic, runs tests, and prepares it for deployment.

Who's involved?

This workflow is designed around 3 liminal roles, directed by a service owner.

1. **The Product Engineer (Viability & Feasibility)** Maps business goals to system architecture. Directs the Agent to connect to rapid BaaS for testing, or build custom infrastructure for scale. Manages the resulting technical debt and CI/CD pipelines.
2. **The Product Designer (Viability & Desirability)** Writes specs defining business rules, logic and Agent constraints. Plans the telemetry approach, and curates the brand design language.
3. **The Design Engineer (Desirability & Feasibility)** Uses MCP tools to sync the canvas and codebase. Prompts the Agent to write target-native code, then uses the IDE to manually polish micro-interactions, performance and accessibility.

The fallback

If your org isn't ready for this yet, the traditional product triad can still adopt the ways of working, so long as all three adopt the "Maker" mindset:

- **Product Manager (Viability):** Begins writing logical specs, instead of tickets, to instruct the Agent.

- **Experience Designer (Desirability):** Masters MCP, and designing-in-code.
- **Software Engineer (Feasibility):** Treats AI as a co-author. Shifts from writing boilerplate to managing the structure of the agent's output.

What this means for experience designers

Designers can no longer avoid being makers. Finally, like all craftspeople, they must know their “materials” and learn to work *with* the digital “grain”.

Top skills to build:

- **Directing agents:** Prompting agents, using MCP and setting rules, like accessibility standards, localization, and performance targets.
- **Mechanical sympathy:** Getting the most out of hardware, software and infrastructure by working with their quirks and constraints.
- **Declarative logic:** Tell the agent how to build UI dynamically by thinking in props and states
- **Data binding:** Mapping JSON structures and APIs to your component props so the agent connects real data to the frontend.
- **Behavioural specs:** Writing logic and requirements to instruct Agents and generate test scripts.
- **The command line:** Spinning up local servers, managing packages, navigating the terminal and basic scripting.
- **Version control:** Branching for experimentation and ideation, fixing merge conflicts, and reviewing pull requests.
- **Telemetry:** Defining what success is, and writing the specs for what to track.
- **Debugging:** Reading terminal logs, tracing data, and reprompting the agent when its code fails.
- **Deployment:** Managing CI/CD pipelines and pushing code to hosting platforms.

-
1. Figma is fine for this but in future [Paper](#) is likely to be a better tool due to having a HTML canvas instead of a vector canvas. ↩
 2. Ideally Dart/Flutter, because it can compile down to mobile, web, desktop and embedded experiences. ↩
 3. eg Mixpanel, Hotjar, GTM etc ↩
 4. Such as Figma's code-to-canvas skill, or preferably, Paper's MCP tools ↩